CL-2005: Database Systems
Lab # 15: Intro to SQL Transactions

Objective:

To understand and implement transactions in SQL Server.

Scope:

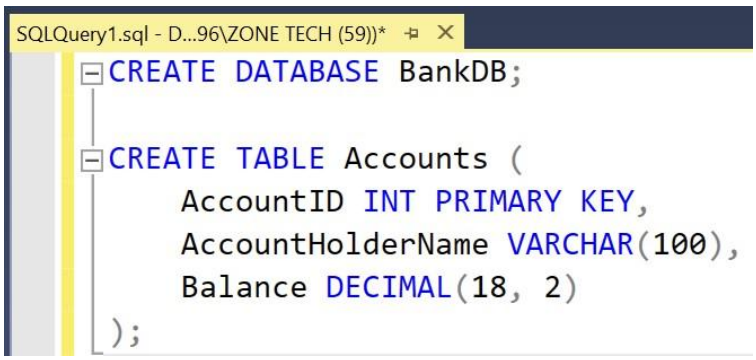
The student shall know the following:

- ACID properties (Atomicity, Consistency, Isolation, Durability)
- COMMIT, ROLLBACK, SAVE Point
- Hands-on experience with the above-mentioned concepts.

Discussion:

A transaction is a logical unit of work. All changes made to the database can be referred to as a transaction. For example, if you are creating, updating or deleting a record from the table, then you are performing a transaction on that table. It is important to control these transactions to ensure the data integrity and to handle database errors.

To understand transactions, lets create BankDB database with an Accounts table;



Properties of Transactions

The database operations in a transaction should follow the following properties referred to as ACID properties:

• **Atomic**: Operations of a transaction are executed all-or-nothing, and are never left "half-done"

• **Consistency**: The transaction should leave the database in a consistent state or we can say a transaction as a whole does not violate integrity constraints.

• **Isolation**: Transactions must behave as if they were executed in complete isolation from each other. One transaction shouldn't affect other transactions running at the same time.

• **Durability**: After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Transactional Control Commands

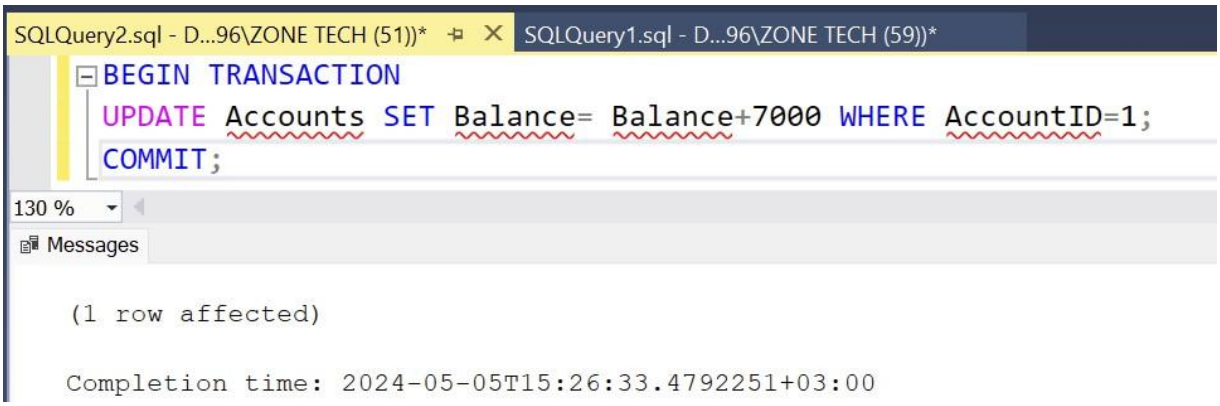
Transactional control commands are only used with the **DML Commands** such as - INSERT, UPDATE and DELETE. They cannot be used while creating tables or dropping them because these operations are automatically committed in the database. Following commands are used to control transactions.

COMMIT:

This command is the transactional command used to save changes invoked by a transaction. With commit command transaction changes can be made permanent to the database.

Syntax: COMMIT;

Example: In SQL commit and rollback works only within transaction, statements outside Transaction is automatically committed to the database.



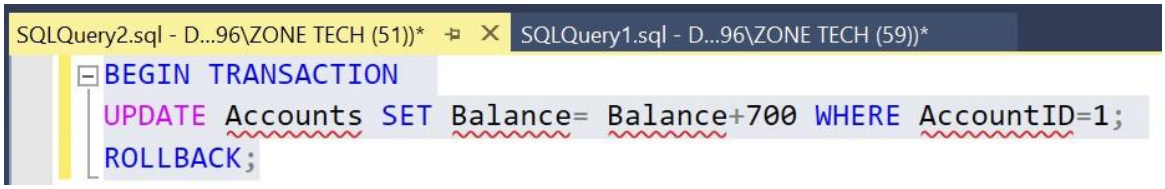
If you will not provide commit in the end of transaction, changes will not be made permanent to database. So, in case your application crashes then these changes will not be reflected in database.

ROLLBACK:

The ROLLBACK command is the transactional command used to undo transactions that have not already been saved to the database. This command can only undo transactions since the last COMMIT or ROLLBACK.

Syntax: ROLLBACK;

Example: Remember if you try to rollback after committing the changes then those changes will not be rolled back.



Try to update some record in your table and roll back. Then check the record in the table, the changes wouldn't be updated there.

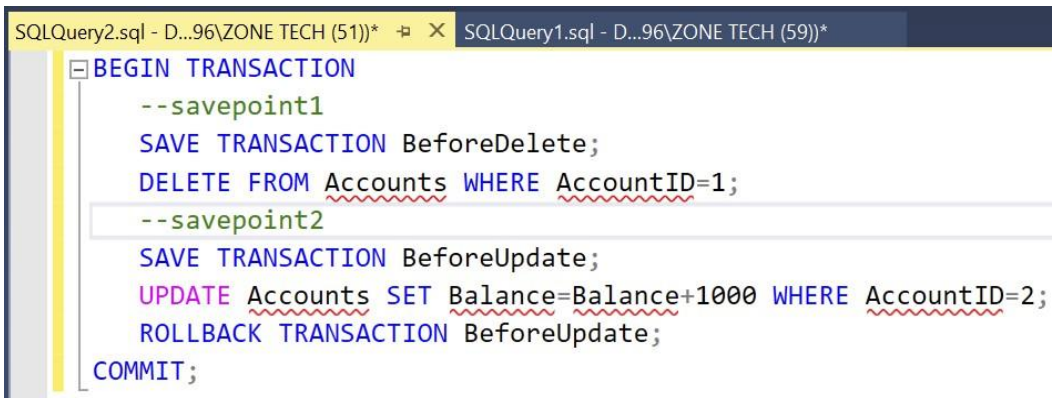
SAVE POINT:

A SAVE point is a logical rollback point in a transaction. They are used to identify a point in a transaction to which we can later rollback.

Syntax: SAVE TRANSACTION savepoint_name;

To roll back to the SAVEPOINT created: ROLLBACK TRANSACTION savepoint_name;

Example: The below query will delete the Account with AccountId=1 but it will not update the record with AccountId=2 as the update statement is rollbacked. If we ROLLBACK transaction to **BeforeDelete** savepoint then both of the statements would be undone.



Error Handling in Transactions

Error handling is crucial in SQL transactions to maintain data integrity and reliability. It ensures that transactions either complete successfully, making necessary changes to the database, or roll back entirely if an error occurs, preventing data inconsistency.

Consider a scenario where a customer wants to transfer money from their account to another account. This process involves deducting the transferred amount from the sender's account and crediting it to the recipient's account. However, various errors such as insufficient balance may occur during this transaction. Therefore, implementing error handling mechanisms is crucial to ensure the atomicity and reliability of the transaction.

The below **procedure TransferMoneyy** checks for all possible errors if the account IDs provided as parameters exist in the Accounts table, if the Senders account has sufficient balance for the transfer, and if any errors occur during the transaction, the procedure raises an appropriate error.

- If the Senders account doesn't have the sufficient amount then the amount should not be transferred to the recipient's account.
- If the Recipient account does not exist then the amount should not be deducted from the Senders account.

SQLQuery3.sql - D...96\ZONE TECH (61))* | SQLQuery2.sql - D...96\ZONE TECH (63))* | SQLQuery1.sql - D...96\ZONE TECH (64))*

```sql
CREATE OR ALTER PROCEDURE TransferMoneyy @FromAccountID INT, @ToAccountID INT, @Amount DECIMAL(18, 2)
AS
BEGIN

BEGIN TRY
    BEGIN TRANSACTION;

        -- Check if Sender's Account exists
        IF NOT EXISTS (SELECT 1 FROM Accounts WHERE AccountID = @FromAccountID)
        BEGIN
            THROW 50002, 'Sender Account does not exist.', 1;
        END
        -- Check if Receipient's exists
        IF NOT EXISTS (SELECT 1 FROM Accounts WHERE AccountID = @ToAccountID)
        BEGIN
            THROW 50003, 'Receipient Account does not exist.', 1;
        END

    -- Check if Sender Account has Sufficient Balance
    DECLARE @FromAccountBalance DECIMAL(18, 2);
    SELECT @FromAccountBalance = Balance FROM Accounts WHERE AccountID = @FromAccountID;

    IF @FromAccountBalance < @Amount
    BEGIN
        THROW 50001, 'Transaction failed: Insufficient balance.', 1;
    END

    -- Deduct Amount from From Account
    UPDATE Accounts SET Balance = Balance - @Amount WHERE AccountID = @FromAccountID;
    -- Add Amount to To Recipient's Account
    UPDATE Accounts SET Balance = Balance + @Amount WHERE AccountID = @ToAccountID;
    -- Commit Transaction
    COMMIT;

    PRINT 'Transfer successful.';
END TRY
BEGIN CATCH
    -- Rollback the transaction if an error occurs
    IF @@TRANCOUNT > 0
    BEGIN
        ROLLBACK;
    END;

    -- Display error message
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH;
END;


EXEC TransferMoneyy @FromAccountID = 1, @ToAccountID = 2, @Amount = 10000.00;
```

TASK

For the Task you need to create a BookStore Database with the following tables

```
CREATE TABLE Books (
   BookID  INT  PRIMARY  KEY,
   Title VARCHAR(255),
   AuthorName    VARCHAR(100),
   Price DECIMAL(10,2),
   StockQuantity INT
);

CREATE TABLE Customers (
   CustomerID INT PRIMARY KEY,
FirstName VARCHAR(100),
LastName VARCHAR(100),
Email VARCHAR(255)
);

CREATE TABLE Orders (
   OrderID  INT  PRIMARY  KEY,
   CustomerID INT,
   OrderDate DATE,
   TotalAmount    DECIMAL(10,2)
);
```

Now insert at least 5 records in each table and perform the following tasks

1. **Write an SQL transaction to update the email address of a specific customer and commit the transaction.**

2. **A customer with CustomerId 43 is ordering the book titled "The Great Gatsby". Write an SQL transaction to insert this order into the database. Use a savepoint before inserting the order details and commit after successful insertion.**

3. **Suppose a customer wants to purchase a book, write a SQL transaction to process this purchase. Create a stored procedure “PurchaseBook” to handle the transaction. If an error occurs due to insufficient stock quantity of the book or due to any other reason, rollback the transaction and print an error message.**